Cloud-Based Distributed Approach for Procedural Terrain Generation with Enhanced Performance

Roman Moravskyi
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
roman.moravskyi.mnpzm.2021@lpnu.ua

Pavlo Pustelnyk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
pavlo.pustelnyk.mnpzm.2021@lpnu.ua

Mykola Morozov
Department of Informatics
Technical University of Munich
Munich, Germany
mykola.morozov@tum.de

Yevheniya Levus
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
yevheniia.v.levus@lpnu.ua

2023 IEEE 18th International Conference on Computer Science and Information Technologies (CSIT) | 979-8-3503-6046-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/CSIT61576.2023.10324223

***Abstract* – Procedural terrain generation has already gained wide acceptance, finding applications across various domains such as video games, films, and simulations. As the demand for content size, quantity, and enhanced fidelity increases, traditional methods prove inadequate for accomplishing this task. Moreover, the planetary scale introduces further constraints on data consistency and distinctiveness. Software systems used for near-real-time procedural terrain generation of large virtual worlds must fulfill high computing requirements. This paper proposes a novel cloud-based approach to execute software agent generation requests with minimized latency. The performance is improved through vertical parallelism using Morsels and horizontal parallelism using multithreading. Apart from spreading request processing per cloud computation instance, performance is enhanced through generation request distribution using a message broker.**

Keyword – software agents; message brokers; gRPC technology; system performance; computational model.

I. INTRODUCTION

Every year, the digital content creation industry needs more realistic results for generating large-scale virtual terrain, provided that the possible number of computations does not exceed the limits of computing. Therefore, it is necessary to develop new landscape models and adapted algorithms for generating spherical virtual worlds. The widespread use of virtual worlds can be seen in cinema and video games [1], particularly planetoids, for creating, storing, and displaying large landscapes. Software-generated planetoids are used for entertainment, science, and education.

Images of entire planets with realistic landscapes are used in scientific research to model various natural processes in three-dimensional virtual worlds [2]. In addition, planetoids are used in architectural modeling [3], including considering the planet's curvature.

Generating planetoids is complex and operates with different types of data containing all the information about the landscape. These include, for example, raster data used for elevation mapping, vector data that reflects the structure of the landscape, the location of rivers and lakes, and the actual three-dimensional objects that fill the virtual world [4].

The state-of-the-art procedural generation of individual landscapes or entire planetoids is determined by the limited number of software tools that provide the necessary tools that are understandable to ordinary users and create sufficiently realistic surfaces. It is essential to solve the problem of ensuring the uniqueness and scalability of the results of the procedural generation of planetoid landscapes in terms of efficient use of computing resources.

II. ANALYSIS AND PROBLEM STATEMENT

Although generating virtual worlds has generally reached a good level of detail and realism, the ever-increasing requirements for quality and performance of visualization dictate the need to develop mathematical models, algorithms, and software implementation methods in procedural landscape generation.

The authors of works in this area solve the problem of generating landscapes using various methods. For instance, the authors of the work [5] aimed to enhance GIS data processing by introducing a flexible model that separates algorithm development, business processes, and operational interfaces, enabling scalable expansion and improved efficiency. The work of B. Fanini and S. Pescarin [6] proposes to establish a cloud-based modular architecture and web services to generate interactive 3D landscapes from geospatial data. On the contrary, there are also studies [7] that aim to develop a serverless Cloud GIS framework for land valuation enabling effective land value data visualization.

However, the methods described above must consider modern hardware and data size, only using network streaming data to mitigate memory limitations [8]. On the one hand, [9] proposes to use cloud computing and big data techniques to address geospatial data challenges. On the other hand, [10] offers to split requests into smaller parallel computing chunks to reduce uneven memory access and network latency.

Given the shortcomings of the considered methods, there is a need to develop a computing system model for procedural landscape generation that would meet the scalability,

979-8-3503-6046-2/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:45:57 UTC from IEEE Xplore. Restrictions apply.

uniformity, and performance requirements for the landscape generation process on a planet scale.

III. PROPOSED SOLUTION

In the proposed solution, the software system's architecture is based on a concept that combines a distributed computing system [10], a software agents approach [11], and a parallelism approach based on the Morsel-Driven Query Execution algorithm [12] to overcome hardware limitations.

The server architecture for generating planetoids is a comprehensive concept for a system that contains all the necessary components and provides an access interface for client applications. The computational model includes the following features:

- The central server that processes gRPC [13] requests from clients;
- Message broker that implements a communication channel between the central server and worker servers.
- Distributed worker servers that simultaneously consume messages from the central server, allowing for parallel cloud computing with performance balanced by the amount of allocated resources. These messages represent requests to execute particular software agents.

The server architecture consists of several physically loosely coupled servers interacting with the database, including a central server that listens for client requests and prepares responses [14]. The other servers use background workers to consume generation tasks from the message broker and execute them. A background worker, which is implemented using the Morsel-Driven Query Execution algorithm, is a process that constantly runs on the server, waits for new messages to appear, and processes them when one of them is published. Using a message broker instead of a database priority queue allows distributing the resource-intensive execution of generating planetoids to several servers in a cloud environment. The combination of Morsels running in the cloud allows for lower message consumption delays.

It is proposed to use a distributed messaging system using Apache Kafka as implementation – a message broker [15] – to send and process requests. Distributed messaging systems allow senders to transmit messages asynchronously regardless of how many recipients exist and where they are located. Furthermore, compared to traditional database storage for messages with a single request-handling component, a message broker can have multiple distributed handlers. In the developed software system, messages are requests to generate or process a given segment of the planetoid landscape by a single agent, and the services that work with these messages are divided into two categories:

1. Producer – is the main microservice with which client applications interact (gRPC or REST requests for segment generation). It publishes messages to "topics" (named logical channels).

2. Consumer – is a microservice that uses background workers whose main task is to process requests for generating or processing segments by specific agents. It reads messages from named logical channels.

An algorithm for proofreading messages based on the priority queue has been developed (Fig. 1) To ensure the correct order of processing the planetoid landscape segment by software agents.

IV. RESULTS

The architectural solution described above is implemented in the PlanetoidGen2 system, which allows generating planetoid landscapes (Fig. 2).

Several computational experiments were conducted to verify the obtained results using different input parameters: the number of background workers, the raster size of the segment in pixels, and the number of segments to be processed (Table 1). The server used: AMD Ryzen 7 5800H CPU (1 CPU, 16 logical and 8 physical cores), Nvidia GeForce RTX 3060 GPU, 16GB DDR4 RAM, NVME2 2TB SSD drive, Windows 11. The experimental environment is Docker Desktop 4.17.0 with a built-in Kubernetes 1.25.4 cluster using Linux containers. Message broker configuration: containerized single-broker Kafka instance (version 3.3.2, Scala version – 2.13, without ZooKeeper) based on the OpenJDK 11 runtime, which is deployed to experimental Kubernetes environment.

The average segment generation time for the above experiments ranges from 0.33 to 9.32 seconds. The lowest average time for generating an individual segment is for a segment size of 512 pixels and the number of segments of 64 (experiments 5, 13, 9, 1) – from 0.33 to 0.51 seconds. The average generation time is increased approximately twice for the exact size of the generated piece but a smaller total number of segments (16). This increase could be explained by a feature of a message broker – a more significant number of

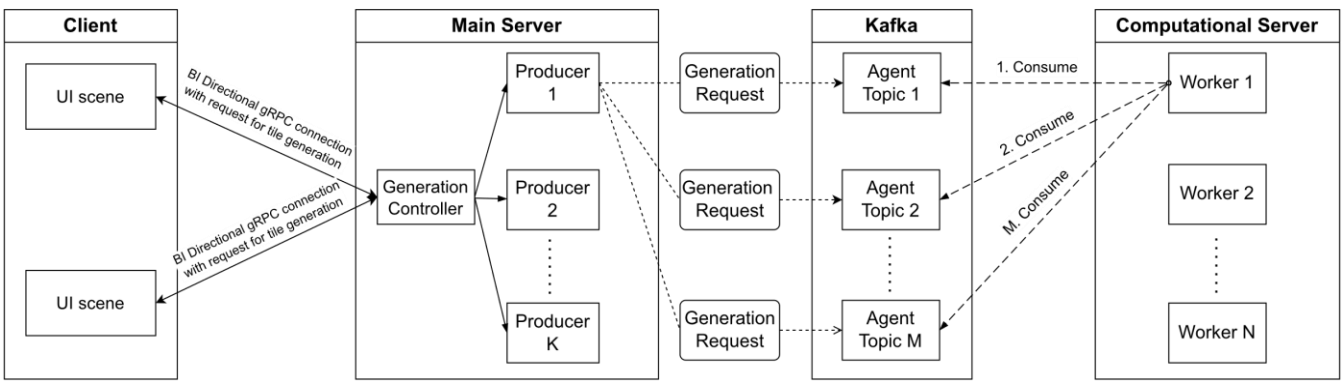


Fig. 1. Diagram of the message reading based on the priority queue.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:45:57 UTC from IEEE Xplore. Restrictions apply.

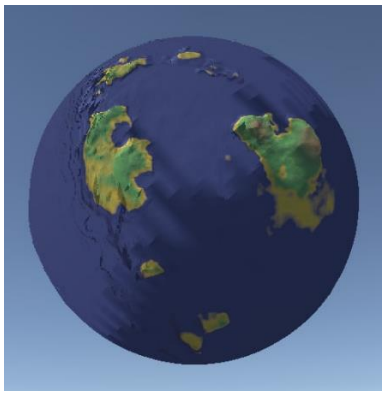


Fig. 2. Generated planet landscape in the PlanetoidGen2 system.

simultaneous generation requests minimizes the average waiting time for reading messages. However, in experiment 17, we can observe the opposite situation: the average generation time is more than two times higher than in the previously described experiments. This anomaly is caused due to the limited number of consumer services that simultaneously read messages (requests) when using the broker. A maximum of 16 consumers can access the channel simultaneously without restructuring the consumer group [16]. The highest average time for a segment size of 512 pixels was recorded for experiments 14 (1.18 seconds) and 18 (1.34 seconds).

For the segment size of 2048 pixels, the lowest average time for generating an individual segment is 2.78 seconds and is observed in Experiment 19 with the number of segments - 64 and background workers – 16. This result is achieved because a more significant number of background workers helps to distribute better a load of generating a large amount of data with minimal broker delays, given the number of generation requests. The highest average time for a segment size of 2048 pixels was recorded for experiments 3 (7.42 seconds) and 4 (9.32 seconds), where one background worker was used.

When using the message broker, the dependence of the average time of generating landscape segments on the number of background workers is not deterministic, unlike the PlanetoidGen1 system [11], which uses a different approach to processing generation requests – saving requests to the database. Non-determinism appears if, when reading messages from logical channels, the number of simultaneous consumer services at a certain point in time approaches the number of batches [16] and, as a result, delays occur due to the restructuring of the consumer group within the message broker. Using a distributed messaging system (broker) made it possible to minimize the time spent on database queries and operations to calculate the priority of the request. The proposed solution allowed us to strengthen the advantages over the MapGen system.

We compared the performance of the developed system with a similar MapGen system [17], which also uses software agents, and a previous version of the PlanetoidGen1 system that does not use a message broker (Table 2).

CONCLUSION

The problem of improving the performance of the process of procedural generation of landscapes for planetoids has been solved based on the constructed computational model, which significantly improves upon the previous research with the following improvements:

- Distributed generation architecture with software agent and message broker nodes uploaded to cloud computing for balanced parallel execution;

TABLE II. Performance comparison with MapGen

System	Segment size, pixels			
	256	*512*	*1024*	*2048*
	Time, s	*Time, s*	*Time, s*	*Time, s*
MapGen	2.08	8.42	49.21	323.01
PlanetoidGen1	1.16	2.98	11.26	47.32
PlanetoidGen2	0.50	0.64	2.14	9.32

TABLE I. Performance of segment generation in PlanetoidGen2

Experiment No	Workers count	Segment size, pixels	Segments count	Total generation time, s	Average generation time for one segment, s
1	1	512	64	32.34	0.51
2	1	512	16	10.20	0.64
3	1	2048	64	474.81	7.42
4	1	2048	16	149.06	9.32
5	2	512	64	21.42	0.33
6	2	512	16	10.20	0.64
7	2	2048	16	104.52	6.53
8	2	2048	64	460.78	7.20
9	4	512	64	27.96	0.44
10	4	512	16	12.86	0.80
11	4	2048	64	269.74	4.21
12	4	2048	16	77.52	4.84
13	8	512	64	26.57	0.42
14	8	512	16	18.89	1.18
15	8	2048	16	78.51	4.91
16	8	2048	64	466.75	7.29
17	16	512	64	72.26	1.13
18	16	512	16	21.44	1.34
19	16	2048	64	178.23	2.78
20	16	2048	16	76.20	4.76

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:45:57 UTC from IEEE Xplore. Restrictions apply.

- Asynchronous processing of generation requests based on Morsel-Driven Query Execution and streaming data for lower processing delays;
- Extensible unified software agent interface with support for loading and saving data to relational tables of any structure;
- A distributed message-broker-based communication approach, replacing a single entry point monolithic query database.

The solution is implemented in the form of the PlanetoidGen2 software system. Computational experiments with different values of the number of background workers, the number and size of segments showed 200%-500% improvement in generation time of one landscape segment compared to the previous version of PlanetoidGen1, which used saving queries to the database rather than a distributed messaging system. Both total and single segment generation time is improved as well, boosting the overall processing speed.

The results show the research prospects and indicate the need for further development. Future research is aimed at enhancing existing or developing new computational models and improving the landscape generation processes based on a distributed computing system to reduce generation time delays. In particular, considering the issue of system scalability with an increase in the amount of generated data, a possible solution to this is the use of message batches or an improved message queue algorithm.

REFERENCE

[1] T. Gao and J. Zhu, "A survey of procedural content generation of natural objects in games," 2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIC), pp. 275–279, 2022. doi: 10.1109/icaiic54071.2022.9722677.

[2] M. Schlecker et al., "The New Generation Planetary Population Synthesis (NGPPS)," Astronomy & Astrophysics, vol. 656, Dec. 2021. doi: 10.1051/0004-6361/202140551.

[3] G. Poyck, "Procedural City Generation with Combined Architectures for Real-time Visualization," M.S. thesis, Dept. Digit. Prod. Art., Clemson Univ., Clemson, SC, USA, 2023.

[4] E. Galin et al., "A review of Digital Terrain Modeling," Computer Graphics Forum, vol. 38, no. 2, pp. 553–577, 2019. doi: 10.1111/cgf.13657.

[5] J. Zhang et al., "An on-demand scalable model for Geographic Information System (GIS) data processing in a cloud GIS," ISPRS International Journal of Geo-Information, vol. 8, no. 9, p. 392, 2019. doi: 10.3390/ijgi8090392.

[6] B. Fanini, S. Pescarin, and A. Palombini, "A cloud-based architecture for processing and dissemination of 3D Landscapes Online," Digital Applications in Archaeology and Cultural Heritage, vol. 14, 2019. doi: 10.1016/j.daach.2019.e00100.

[7] M. O. Mete and T. Yomralioglu, "Implementation of serverless cloud GIS platform for Land Valuation," International Journal of Digital Earth, vol. 14, no. 7, pp. 836–850, 2021. doi: 10.1080/17538947.2021.1889056.

[8] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, "LogPlayer: Fault-tolerant Exactly-once Delivery using gRPC Asynchronous Streaming," ArXiv, Nov. 2019, doi: 10.48550/arXiv.1911.11286.

[9] C. Yang, M. Yu, F. Hu, Y. Jiang, and Y. Li, "Utilizing cloud computing to address Big Geospatial Data challenges," Computers, Environment and Urban Systems, vol. 61, pp. 120–128, 2017. doi: 10.1016/j.compenvurbsys.2016.10.010.

[10] R. H. Jacobsen et al., "A scalable cloud computing infrastructure for Geospatial Data Analytics for Change Detection," 2017 Euromicro Conference on Digital System Design (DSD), pp. 403–410, 2017. doi: 10.1109/dsd.2017.49.

[11] Y. Levus, R. Westermann, M. Morozov, R. Moravskyi, and P. Pustelnyk, "Using software agents in a distributed computing system for procedural planetoid terrain generation," 2022 IEEE 17th International Conference on Computer Sciences and Information Technologies (CSIT), pp. 446–449, 2022. doi: 10.1109/csit56902.2022.10000868.

[12] K. Dursun, C. Binnig, U. Cetintemel, G. Swart, and W. Gong, "A morsel-driven query execution engine for heterogeneous multi-cores," Proceedings of the VLDB Endowment, vol. 12, no. 12, pp. 2218–2229, 2019. doi: 10.14778/3352063.3352137.

[13] E. R. Jamzuri, H. Mandala, R. Analia, and S. Susanto, "Cloud-based architecture for Yolov3 Object Detector using gRPC and Protobuf," Jurnal Teknik Elektro, vol. 14, no. 1, pp. 18–23, 2022. doi: 10.15294/jte.v14i1.36537.

[14] R. Minailenko, O. Sobinov, O. Konoplitska-Slobodenyuk, and K. Buravchenko, "Architectural features of Distributed Computing Systems," Central Ukrainian Scientific Bulletin. Technical Sciences, no. 4(35), pp. 16–23, 2021. doi: 10.32515/2664-262x.2021.4(35).16-23.

[15] T. Sharvari and K. Sowmya Nag, "A study on Modern Messaging Systems – Kafka, RabbitMQ and NATS Streaming," ArXiv, Dec. 2019, doi: 10.48550/arXiv.1912.03715.

[16] H. Wu, Z. Shang, and K. Wolter, "Performance prediction for the Apache Kafka Messaging System," 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), pp. 154–161, 2019. doi: 10.1109/hpcc/smartcity/dss.2019.00036.

[17] J. Doran and I. Parberry, "Controlled procedural terrain generation using software agents," IEEE Transactions on Computational Intelligence and AI in Games, vol. 2, no. 2, pp. 111–119, 2010. doi: 10.1109/tciaig.2010.2049020.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on June 10,2024 at 11:45:57 UTC from IEEE Xplore. Restrictions apply.